1. What are higher order functions?
   The functions which accept another function or returns a function is known as higher order function.

```
var arr = [1,2,3];
arr.forEach(function() {});
```

   Here forEach function is higher order function.

2. What are constructor functions?
   Constructor functions are basically the blueprint for creating many objects of same type.
   To create objects of same type you just need to call the construction function. By using new keyword.

```
function Person(first, last, age, eye) {
   this.firstName = first;
   this.age = age;
}
```

   Example: const human1 = new Person("ABD", 21);

3. What is this?
   In JavaScript, the this keyword refers to an object.

   Which object depends on how this is being invoked (used or called).

   The this keyword refers to different objects depending on how it is used:

   In an object method, this refers to the object.
   Alone, this refers to the global object(Window).
   In a function, this refers to the global object(Window).
   In a function, in strict mode, this is undefined.
   In an event, this refers to the element that received the event.

4. What is new keyword?
   New keyword in JavaScript is used to create an instance of an object that has a constructor function. On calling the constructor function with ‘new’ operator, the following actions are taken:

   1. A new empty object is created.
   2. The ‘this’ variable is made to point to the newly created object. It binds the property which is declared with ‘this’ keyword to the new object.

   3. About the returned value, there are three situations below.
      If the constructor function returns a non-primitive value (Object, array, etc), the constructor function still returns that value. Which means the new operator won’t change the returned value.
      If the constructor function returns nothing, ‘this’ is return;
      If the constructor function returns a primitive value, it will be ignored, and ‘this’ is returned.

5. What is iife?
   IIFE stands for Immediately Invoked function Expression.
   Basically, IIFE are the functions which gets called as soon as they are created.
   Syntax:

```
(function (){
      // Function Logic Here.
})();
```

   Immediately Invoked: This part is easy to explain and demonstrate. This type of function is called immediately invoked as these functions are executed as soon as they are mounted to the stack, it requires no explicit

call to invoke the function. If we look at the syntax itself we have two pairs of closed parentheses, the first one contains the logic to be executed and the second one is generally what we include when we invoke a function, the second parenthesis is responsible to tell the compiler that the function expression has to be executed immediately.

Function Expressions: It is simple to understand that a Function Expression is a Function that is used as an expression. JavaScript lets us use Functions as Function Expressions if we Assign the Function to a variable, wrap the Function within parentheses or put a logical not in front of a function.

```
// Creating Function Expression by assigning to a variable.
   var myFunc = function() { return "GeeksforGeeks"; };

// Creating Function Expression Using Logical Not.
   !function() { return "GeeksforGeeks"; };

// Creating Function Expression Using Parentheses.
   (function() { return "GeeksforGeeks"; });
```

6. What is prototype?

Whenever we create objects in javascript, js automatically adds some properties to it, this is only known as prototype.

Prototype contains many helper properties and methods we can use to complete our task, let's say we create an array and we want to know length of it, what do we do, we use .length property on array. This .length property is provided by prototype.

Many properties and methods are already available to use built by js creators inside prototype of every object.

7. What is prototype inheritance?

Prototype inheritance in javascript is the linking of prototypes of a parent object to a child object to share and utilize the properties of a parent class using a child class. Prototypes are hidden objects that are used to share the properties and methods of a parent class to child classes.

```
child.__proto__ = parent;
```

8. Difference between method and function?

A function within the object is known as method.

```
var obj = {
            abcd: function() {};  // Method
        }
```

9. call, apply, bind

We know that when we create an object, this is already related to something. But when we want this to refer something explicitly we use call, apply and blind.

call(), apply()

The official docs for call() say: The call() method calls a function with a given "this" value and arguments provided individually.

What that means, is that we can call any function, and explicitly specify what this should reference within the calling function. Really similar to the bind() method!

The main differences between bind() and call() is that the call() method:

Accepts additional parameters as well
Executes the function it was called upon right away.
The call() method does not make a copy of the function it is being called

on.
call() and apply() serve the exact same purpose. The only difference between how they work is that call() expects all parameters to be passed in individually, whereas apply() expects an array of all of our parameters.

Example:

```
var pokemon = {
    firstname: 'Pika',
    lastname: 'Chu ',
    getPokeName: function() {
    var fullname = this.firstname + ' ' + this.lastname;
    return fullname;
    }
};

var pokemonName = function(snack, hobby) {
    console.log(this.getPokeName() + ' loves ' + snack + ' and ' + hobby);
};

pokemonName.call(pokemon,'sushi', 'algorithms'); // Pika Chu  loves sushi and algorithms
pokemonName.apply(pokemon,['sushi', 'algorithms']); // Pika Chu loves sushi and algorithms
```

bind()

The official docs say: The bind() method creates a new function that, when called, has its "this" keyword set to the provided value.

```
function abcd() {
    console.log(this);
}

var obj = {age : 24};

var bindedFnc = abcd.bind(obj);
```

Basically it binds the function with object and gives you the binded function.

This is basically used in React when we want certain actions to be performed only when certain action has already been performed.

10. Pure Functions

Pure functions are those functions which follow these two conditions:

1. It should always return same output for same input.
2. It will never change or update the value of a global variable.

Impure Functions

Opposite of pure functions.

Example: `function abcd(val) { return Math.random() * val; }`